UNIVERSIDAD PERUANA CAYETANO HEREDIA

FACULTAD DE CIENCIAS E INGENIERÍA
CARRERA DE INGENIERÍA INFORMÁTICA



PROYECTO PARCIAL

Programación Orientada a Objetos

Fuga de la Prisión

Diseño e implementación de un videojuego 2D

Curso:	Programación Orientada a Objetos
Docentes:	MSc. Wilder Nina Choquehuayta (principal) MSc. Percy Lovón Ramos (auxiliar) Dr. Ortiz Ortiz Emanuel (auxiliar)
Tema asignado:	Fuga de la Prisión
Ciclo:	IV
Integrantes:	JOSE FERNANDO CHAVEZ LOZANO MICKELE CARLOS TORRES VELAZCO PIERO CEMEI ROMERO MEZA RICHARD AARON SORIANO CORDOVA
Período académico:	2026-II

Lima — Perú

2026

Informe parcial: Fuga de la Prisión

I. Resumen Ejecutivo

Fuga de la Prisión es un videojuego 2D de escape y sigilo con cuatro niveles y un jefe. La arquitectura separa personajes, mapa, puertas, ruido, estados, puntaje, combate y dibujo. Sus 25 clases aparecen en el UML y aplican asociación, agregación, composición, encapsulamiento, herencia y polimorfismo. State organiza los estados y Observer comunica el puntaje. El avance incluye una celda inicial, alarmas que alertan a los guardias y un jefe con tres capas de vida. Las pruebas y capturas comprueban estas funciones.

II. Introducción

a. Contexto y objetivo

El proyecto evalúa la aplicación de la Programación Orientada a Objetos mediante un videojuego. La temática asignada es Fuga de la Prisión: escapar evitando guardias, cámaras y sistemas de seguridad [1]. La entrega parcial busca demostrar un diseño de clases coherente, sus fundamentos POO y los patrones implementados, acompañados de una interfaz y funciones que se puedan ejecutar [2].

La campaña contiene tres zonas de sigilo y una arena contra el Alcaide. En el primer nivel se empieza encerrado y se fuerza la puerta manteniendo E. Cada zona exige conseguir una llave y alcanzar la salida. Los pasos pueden atraer patrullas; las cámaras alertan a todos los guardias y la captura ocurre por contacto. El último mapa de sigilo tiene galerías horizontales y atajos que se abren con su llave.

b. Tecnologías utilizadas

Tecnología	Uso
Python y Pygame	Reglas, ventana, teclado y dibujo de los personajes stickman. Verificado con Python 3.13.7 y Pygame 2.6.1.
Mermaid.js	Fuente editable del UML e imagen generada desde el mismo modelo.
unittest	Pruebas de juego, celda, alarmas, paredes, campaña, tres capas del jefe y correspondencia entre código y documentos.
Git y GitHub	Registro y publicación de versiones. Se revisa el historial real del repositorio.

III. Diseño de Arquitectura Conceptual y de Clases

a. Diagrama de clases UML

El modelo incluye las 25 clases del juego, conforme a la indicación del docente de representar todas. Cada caja tiene nombre, atributos propios y operaciones. Los atributos que guardan objetos señalan su clase: mapa: Mapa, celda: Optional<PuertaCelda>, ruidos: SistemaRuido, carrera: HabilidadCarrera y jefe: Alcaide. Los miembros heredados se leen en la base. Las propiedades se muestran como atributos; los constructores se incluyen en la guía completa y se omiten en el UML para reducir su tamaño.

El diagrama completo está en la última página ampliada. modelo_parcial.mmd es la fuente editable y modelo_parcial.png se genera con Mermaid.js. Las clases de pruebas no son componentes del videojuego.

b. Estructura de componentes

Archivo y clases	Responsabilidad
juego.py: Juego	Coordina la campaña, conserva puntos y delega en el estado actual.
estados.py: EstadoJuego, Jugando, Pausa, Derrota, Victoria, NivelCompletado	Definen entrada, actualización, pausa y transición entre niveles.
nivel.py: Nivel, PuertaCelda, Camara, Llave, PuertaSalida	Coordinan la celda, alertas, captura, recogida de llaves, atajos, premios y salida.
mapa.py: Mapa	Conserva el plano, abre puertas, resuelve colisiones y busca caminos.
personajes.py: Personaje, Prisionero, Guardia, HabilidadCarrera	Movimiento común, controles, patrulla, investigación, persecución y carrera.
ruido.py: SistemaRuido	Registra a los guardias y les notifica los pasos que llegan por los pasillos.
puntuacion.py: Puntaje, ObservadorPuntaje, PanelPuntaje	Validan los puntos y comunican sus cambios mediante Observer.
combate.py: Proyectil, Alcaide, Combate	Disparos, tres capas de vida, fases, avisos, impactos e inmunidad.
vista.py: Vista	Dibuja mapas, celda, cámaras ancladas, ruido, alertas y barras del jefe.

Lectura de las relaciones

El rombo negro representa composición y queda junto al objeto que crea sus partes; el blanco representa agregación. El triángulo vacío apunta a la clase base. La flecha continua indica una referencia guardada y la discontinua un uso o dependencia. 1, 0..1 y 0..* significan uno, ninguno o uno, y ninguno o varios. + indica acceso público; # señala uso interno por convención de Python.

IV. Aplicación de Fundamentos de POO

a. Asociación, agregación y composición

Asociación. Personaje recibe y guarda su Nivel para consultar datos y colisiones. SistemaRuido conserva el Mapa recibido, sin crear otro:

```
self.nivel = nivel
self.mapa = mapa
```

La primera instrucción pertenece a Personaje y la segunda a SistemaRuido.

Agregación. Juego crea Puntaje y lo entrega al Nivel. El mismo objeto se conserva al cambiar de nivel. SistemaRuido también guarda guardias que Nivel creó y registró como oyentes:

```
self.nivel = Nivel(self.puntaje, self.indice_nivel)
self.ruidos.registrar(guardia)
```

Composición. Nivel crea su Mapa, SistemaRuido, personajes, cámaras, llave y salida. Solo en el primer nivel crea PuertaCelda; en la arena crea Combate. Prisionero crea HabilidadCarrera y Combate crea Alcaide y Proyectil:

```
self.mapa = Mapa(PLANOS[indice])
self.celda = PuertaCelda(centro(4, 11)) if indice == 0 else None
```

b. Herencia y polimorfismo

Prisionero y Guardia heredan de Personaje. Comparten posición y movimiento, pero cada uno implementa update: el prisionero responde al teclado y emite pasos; el guardia percibe, decide y sigue caminos. Nivel utiliza una llamada común:

```
for personaje in self.personajes:
    personaje.update()
```

Juego delega actualizar y manejar_entrada en su EstadoJuego activo. Los cinco estados concretos responden según la situación. PanelPuntaje hereda de ObservadorPuntaje e implementa actualizar_puntaje.

c. Encapsulamiento y abstracción

Puntaje conserva _valor y lo modifica mediante set_valor, sumar y reiniciar. El setter valida el dato y notifica el cambio. Personaje consulta sus coordenadas mediante get_posicion y las modifica con mover, que respeta las colisiones. El guion bajo expresa uso interno; no impide por sí solo el acceso en Python.

```
def get_valor(self):
    return self._valor

def set_valor(self, valor):
    if type(valor) is not int or valor < 0:
        raise ValueError("El puntaje debe ser un entero no negativo.")
    self._valor = valor
    self._notificar()
```

Personaje, EstadoJuego y ObservadorPuntaje son abstractas: sus clases hijas deben implementar las operaciones indicadas. Esto permite compartir una base y conservar el comportamiento propio de cada objeto.

Patrones de diseño implementados

State. EstadoJuego es la base de Jugando, Pausa, Derrota, Victoria y NivelCompletado. Juego delega en el estado activo. La pausa detiene movimiento, celda, vigilancia, temporizadores y combate.

Observer. Puntaje notifica a PanelPuntaje al cambiar su valor. SistemaRuido utiliza una comunicación semejante: registra oyentes y llama escuchar cuando les llega un paso; el guardia decide su conducta.

VI. Implementación de Resultados Parciales

Se juega con WASD o flechas, Ctrl reduce el ruido y Shift permite correr. Al inicio se debe mantener E durante dos segundos junto a los barrotes para salir de la celda. Las cámaras están alineadas con las paredes: su alarma alerta a todos los guardias y no causa derrota instantánea. Los guardias investigan, persiguen y buscan en la última posición conocida; solo capturan por contacto.

El Patio tiene tres galerías horizontales, tres patrullas y tres cámaras. La llave del ala norte abre dos atajos de regreso. Las puertas cambian tanto el dibujo como las colisiones y los caminos. En la arena, el Alcaide dispone de tres capas de 72 puntos: la llave aparece únicamente tras agotar las tres.

Fase	Coronas	Velocidad máxima de la corona	Espera tras atacar
1: tres capas restantes	12 proyectiles	115 píxeles/s	1,45 s
2: dos capas restantes	40 proyectiles	155 píxeles/s	0,85 s
3: una capa restante	84 proyectiles	205 píxeles/s	0,45 s

Las ráfagas y embestidas también aceleran. Cada cambio de capa se anuncia durante 1,1 segundos; se retiran los proyectiles y se evita perder varias capas en el mismo instante.





Figura 1. Celda inicial cerrada. Figura 2. Última capa del jefe, con la fase 3 activa. Capturas del juego ejecutado durante la verificación.

Resultados comprobados y uso de Git

El 15 de octubre de 2026 se ejecutaron 54 pruebas automáticas sin fallos. Cubren celda, apertura de atajos, alarmas, paredes, conectividad, ruido, persecución, captura, puntaje, estados y combate con tres capas. También comparan las clases, atributos, métodos y código de la guía con el UML. La verificación completó los cuatro niveles mediante movimiento y disparos, sin desactivar enemigos, y generó catorce capturas.

La llave concede 100 puntos, cada salida 500 y el jefe 1000: la campaña termina con 3400. El jefe fue vencido en 72,52 segundos de simulación. R repite el nivel y restaura los puntos de entrada; N empieza desde cero. La rúbrica pide mostrar un avance del 25 % con interfaces y funciones [2], pero no da una fórmula para medirlo; aquí se presentan funciones comprobables.

Git. El código del juego analizado corresponde a 56f629f, con catorce commits. El historial registra planificación, implementación y documentación. Las revisiones del material de exposición y de este informe se incorporan al mismo repositorio. Los nombres configurados de los autores no identifican por sí solos los aportes de los cuatro integrantes; cada uno debe explicar cambios concretos que pueda sustentar.

VII. Conclusiones

El avance reúne cuatro niveles, apertura de celda, sigilo, ruido de pasos, investigación, persecución, llaves, atajos, puntaje y un jefe con tres capas de dificultad creciente. Sus 25 clases aparecen en el UML y están explicadas con su código. Se pueden demostrar los seis fundamentos POO y State y Observer.

La dificultad principal fue mantener coherentes puertas, paredes, sonido y caminos. Separar Mapa, PuertaCelda y SistemaRuido de la coordinación de Nivel permitió comprobar cada regla. Las pruebas de alarmas y cambios de capa evitaron derrotas y premios anticipados. El equipo debe ensayar la partida con teclado y preparar la explicación oral.

VIII. Anexos

Fragmento 1. La apertura de la celda modifica el mismo mapa que utiliza el movimiento:

```
if self.celda and self.celda.actualizar(
        dt, self.prisionero, "interactuar" in self.teclas):
    self.mapa.abrir_puerta(self.celda.posicion)
    self.ruidos.emitir(self.celda.posicion, 260)
```

Fragmento 2. La fase depende de las capas que conserva el jefe:

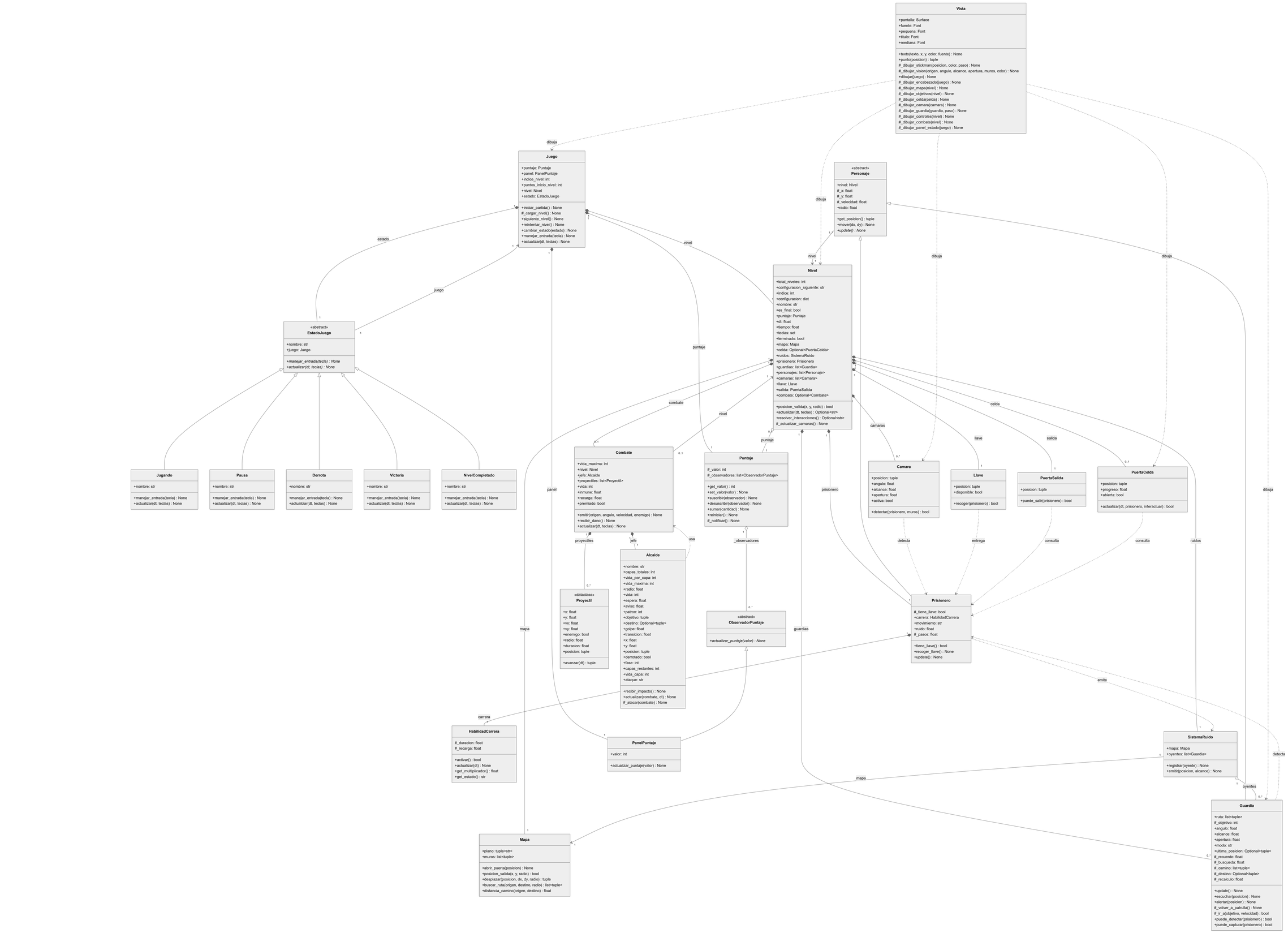
```
return min(self.capas_totales,
           self.capas_totales - self.capas_restantes + 1)
```

El UML completo está en la página siguiente. Los anexos posteriores desarrollan las 25 clases con su recorte del UML, función, aplicación de POO y fragmentos reales del código.

Material del curso

[1] Proyecto Final POO, ciclo 2026-2, temática 13.
[2] Rúbrica y Estructura - Proyecto Parcial, ciclo 2026-2.

VIII. Anexos - Diagrama UML completo: 25 clases del juego



Nombre, atributos propios y operaciones. Propiedades como atributos; miembros heredados en la base.
Rombo negro: composición | Rombo blanco: agregación | Triángulo: herencia | Flechas: asociación y dependencia

VIII. Anexos: explicación de las 25 clases

Cada ficha incorpora la caja exacta de la clase extraída del UML original, su función, los conceptos que realmente aplica y un fragmento del código. Las fichas sustentan los criterios de arquitectura y fundamentos POO de la rúbrica.

Una relación de composición también se explica desde la clase que actúa como parte: por ejemplo, Nivel crea Mapa. Un atributo público no se presenta como si fuera privado. State y Observer se distinguen de los seis fundamentos de POO. El guion bajo en Python es una convención de uso interno.

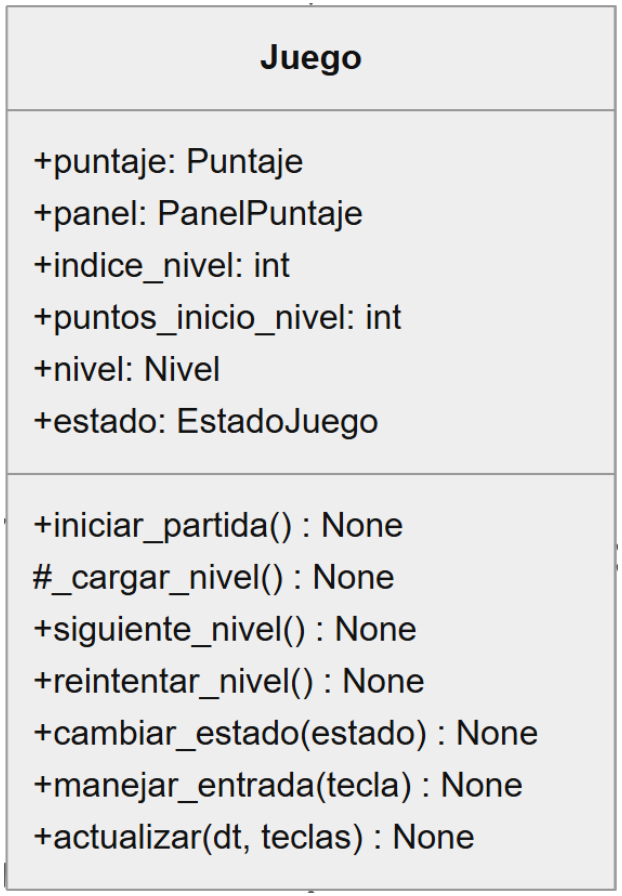
Índice de fichas

01. Juego p. 9
02. EstadoJuego p. 10
03. Jugando p. 11
04. Pausa p. 12
05. Derrota p. 13
06. Victoria p. 14
07. NivelCompletado p. 15
08. Nivel p. 16
09. PuertaCelda p. 17
10. Mapa p. 18
11. Personaje p. 19
12. Prisionero p. 20
13. Guardia p. 21
14. HabilidadCarrera p. 22
15. SistemaRuido p. 23
16. Camara p. 24
17. Llave p. 25
18. PuertaSalida p. 26
19. Puntaje p. 27
20. ObservadorPuntaje p. 28
21. PanelPuntaje p. 29
22. Proyectil p. 30
23. Alcaide p. 31
24. Combate p. 32
25. Vista p. 33

Juego

Coordina la campaña, los niveles, los puntos y el estado de la partida.

UML: recorte de la caja original



Aplicación de POO

Composición: crea Puntaje, PanelPuntaje y Nivel.

Polimorfismo: delega en el EstadoJuego activo.

Código del proyecto (fragmento)

```
class Juego:

    def __init__(self):
        self.puntaje = Puntaje()
        self.panel = PanelPuntaje()
        self.puntaje.suscribir(self.panel)

    def manejar_entrada(self, tecla):
        self.estado.manejar_entrada(tecla)
```

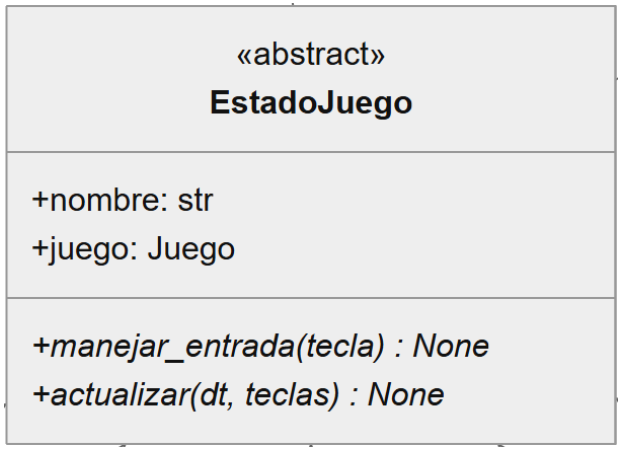
Juego coordina objetos con responsabilidades distintas. La creación de las partes muestra composición. Al llamar al estado activo usa una interfaz común: el comportamiento depende de si ese objeto es Jugando, Pausa u otro estado. La suscripción del panel conecta además el patrón Observer.

Fuente: src/juego.py, clase desde línea 7.

EstadoJuego

Define las operaciones comunes de los estados de la partida.

UML: recorte de la caja original



Aplicación de POO

Abstracción: exige implementar entrada y actualización.

Asociación: conserva el Juego que recibe.

Código del proyecto (fragmento)

```
class EstadoJuego(ABC):

    def __init__(self, juego):
        self.juego = juego

    @abstractmethod
    def actualizar(self, dt, teclas):
        pass
```

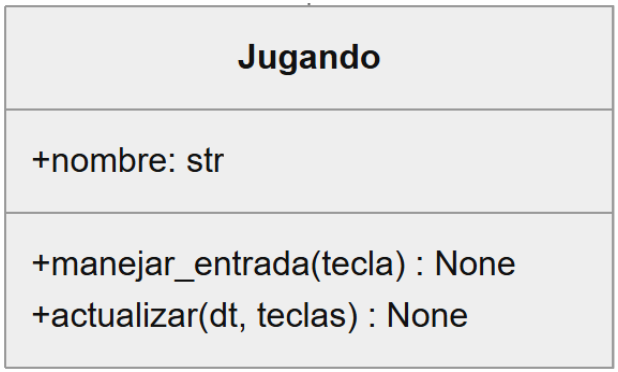
Hereda de ABC y define el contrato de los estados. Los decoradores abstractmethod impiden instanciar una implementación incompleta. Guarda una referencia a Juego sin construirlo. Es la base del patrón State, no un estado concreto que el usuario vea en pantalla.

Fuente: src/estados.py, clase desde línea 4.

Jugando

Mantiene activa la simulación del nivel.

UML: recorte de la caja original



Aplicación de POO

Herencia: deriva de EstadoJuego.

Polimorfismo: actualizar ejecuta la simulación.

Código del proyecto (fragmento)

```
class Jugando(EstadoJuego):

    def actualizar(self, dt, teclas):
        resultado = self.juego.nivel.actualizar(dt, teclas)
        if resultado == "derrota":
            self.juego.cambiar_estado(Derrota(self.juego))
        elif resultado == "victoria":
            self.juego.cambiar_estado(Victoria(self.juego))
        elif resultado == "nivel_completado":
            self.juego.cambiar_estado(NivelCompletado(self.juego))
```

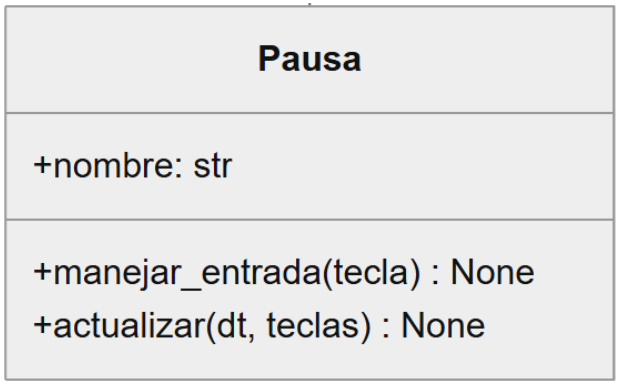
Implementa las operaciones heredadas para una partida activa. Juego puede llamarla por la misma interfaz que usa con Pausa. El resultado del nivel decide la transición a derrota, victoria o nivel completado. Es uno de los estados concretos de State.

Fuente: src/estados.py, clase desde línea 19.

Pausa

Detiene la simulación hasta que el jugador continúa.

UML: recorte de la caja original



Aplicación de POO

Herencia: deriva de EstadoJuego.

Polimorfismo: actualizar deja el nivel detenido.

Código del proyecto (fragmento)

```
class Pausa(EstadoJuego):

    def manejar_entrada(self, tecla):
        if tecla == "pausa":
            self.juego.cambiar_estado(Jugando(self.juego))

    def actualizar(self, dt, teclas):
        pass
```

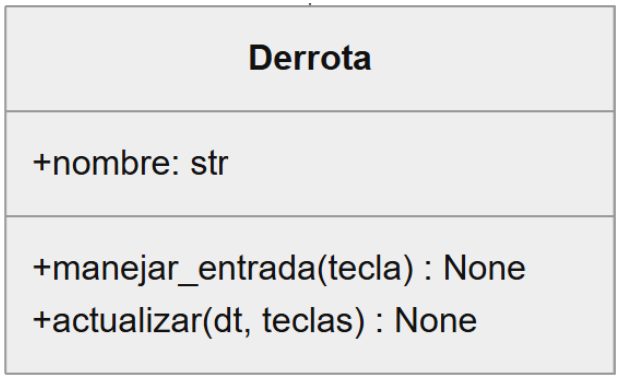
Comparte la interfaz de Jugando, pero su actualización no realiza ninguna operación. Esa diferencia de comportamiento con la misma llamada demuestra polimorfismo. Su método de entrada vuelve a Jugando cuando recibe la acción de pausa.

Fuente: src/estados.py, clase desde línea 38.

Derrota

Espera la decisión del jugador después de perder.

UML: recorte de la caja original



Aplicación de POO

Herencia: deriva de EstadoJuego.

Polimorfismo: la entrada permite reintentar o reiniciar.

Código del proyecto (fragmento)

```
class Derrota(EstadoJuego):

    def manejar_entrada(self, tecla):
        if tecla == "reiniciar":
            self.juego.reintentar_nivel()
        elif tecla == "nueva_partida":
            self.juego.iniciar_partida()

    def actualizar(self, dt, teclas):
        pass
```

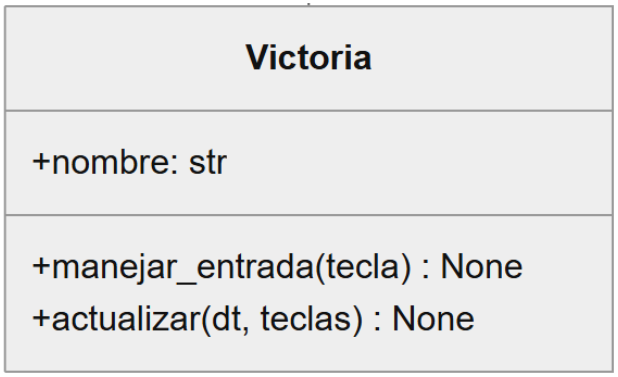
El mismo mensaje manejar_entrada tiene aquí el comportamiento propio de perder. Reintentar restaura el nivel actual, mientras nueva_partida inicia la campaña desde cero. actualizar no hace avanzar la simulación. Participa en State.

Fuente: src/estados.py, clase desde línea 49.

Victoria

Representa el final de la campaña completada.

UML: recorte de la caja original



Aplicación de POO

Herencia: deriva de EstadoJuego.

Polimorfismo: reiniciar inicia una campaña nueva.

Código del proyecto (fragmento)

```
class Victoria(EstadoJuego):

    def manejar_entrada(self, tecla):
        if tecla == "reiniciar":
            self.juego.iniciar_partida()

    def actualizar(self, dt, teclas):
        pass
```

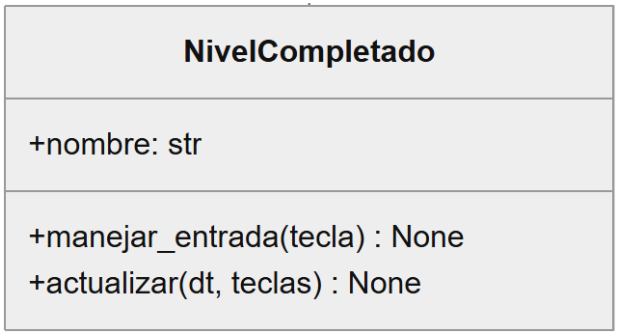
Representa una campaña terminada. Su implementación de entrada interpreta reiniciar como comenzar otra campaña. Su actualización vacía mantiene detenido el nivel final. Las otras clases concretas responden a la misma interfaz con decisiones distintas.

Fuente: src/estados.py, clase desde línea 62.

NivelCompletado

Separa el final de un nivel del final de toda la campaña.

UML: recorte de la caja original



Aplicación de POO

Herencia: deriva de EstadoJuego.

Polimorfismo: continuar carga el siguiente nivel.

Código del proyecto (fragmento)

```
class NivelCompletado(EstadoJuego):

    def manejar_entrada(self, tecla):
        if tecla == "continuar":
            self.juego.siguiente_nivel()
        elif tecla == "nueva_partida":
            self.juego.iniciar_partida()

    def actualizar(self, dt, teclas):
        pass
```

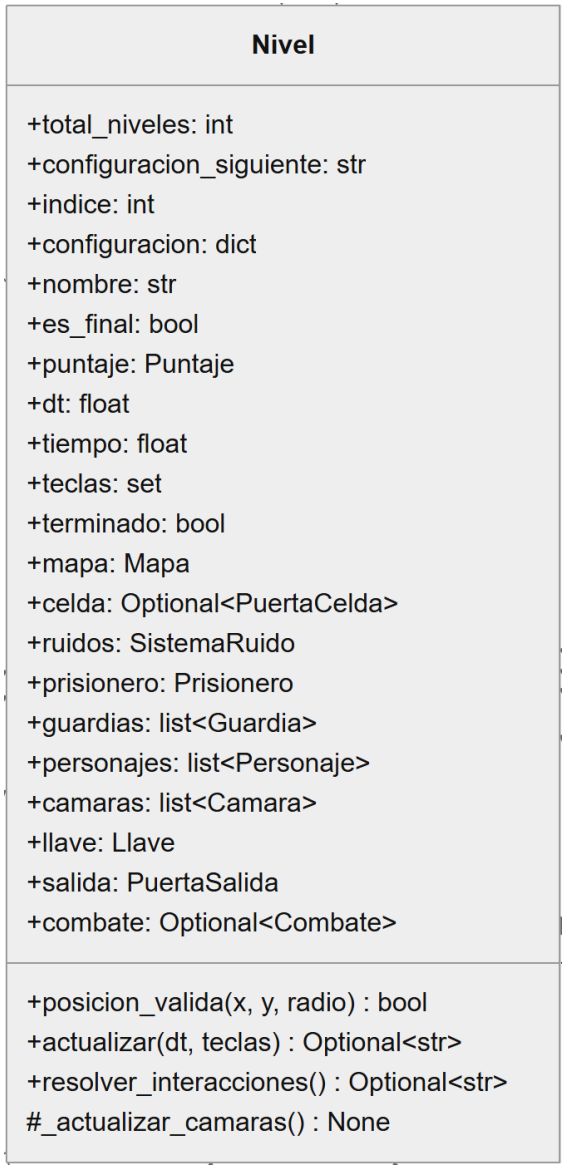
Separa la salida de un nivel de la victoria de toda la campaña. La implementación de manejar_entrada decide cuándo avanzar y cuándo empezar desde cero. No actualiza el nivel mientras espera esa decisión. Es el quinto estado concreto de State.

Fuente: src/estados.py, clase desde línea 73.

Nivel

Coordina uno de los cuatro niveles y resuelve sus interacciones.

UML: recorte de la caja original



Aplicación de POO

Composición: construye Mapa y personajes.

Agregación: recibe el Puntaje compartido.

Polimorfismo: llama update sin distinguir personajes.

Código del proyecto (fragmento)

```
class Nivel:

    def __init__(self, puntaje, indice=0):
        self.puntaje = puntaje
        self.mapa = Mapa(PLANOS[indice])

    def actualizar(self, dt, teclas):
        for personaje in self.personajes:
            personaje.update()
```

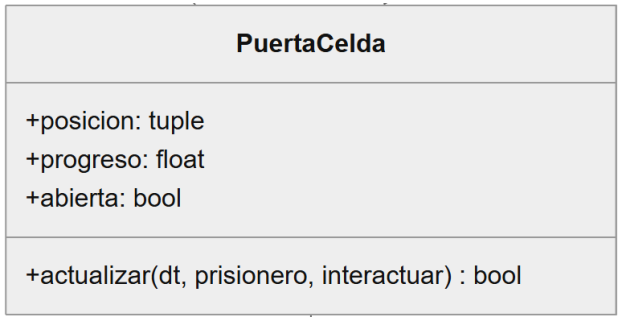
Recibe el Puntaje que Juego conserva entre niveles, pero construye las partes propias de la etapa. En self.personajes conviven Prisionero y Guardia. El recorrido del fragmento no pregunta su tipo: cada instancia ejecuta su versión de update. La composición es una relación del diseño, no una orden de destrucción automática de Python.

Fuente: src/nivel.py, clase desde línea 85.

PuertaCelda

Mantiene cerrada la celda inicial y controla cómo se fuerza su puerta.

UML: recorte de la caja original



Aplicación de POO

Composición: Nivel crea su puerta inicial.

Dependencia: actualizar recibe al Prisionero.

Código del proyecto (fragmento)

```
class PuertaCelda:

    def actualizar(self, dt, prisionero, interactuar):
        if (
            not interactuar
            or distancia(self.posicion, prisionero.get_posicion())
            > 48
        ):
            self.progreso = 0.0
            return False
```

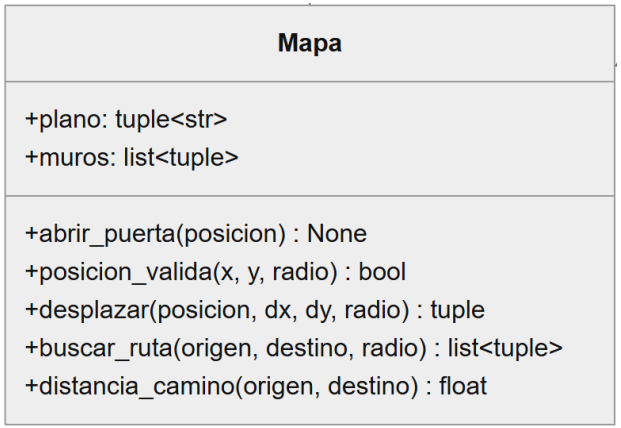
Es una parte de Nivel que controla el progreso de apertura. actualizar consulta la cercanía del Prisionero y la acción de interactuar. Cuando alcanza dos segundos cambia abierta y Nivel retira los barrotes de su mapa. Sus atributos son públicos: no es un ejemplo de acceso restringido mediante getters y setters.

Fuente: src/nivel.py, clase desde línea 32.

Mapa

Conserva el plano y reúne las reglas de paredes y caminos.

UML: recorte de la caja original



Aplicación de POO

Composición: Nivel crea su propio Mapa.

Abstracción: ofrece colisiones y rutas mediante métodos.

Código del proyecto (fragmento)

```
class Mapa:

    def distancia_camino(self, origen, destino):
        ruta = self.buscar_ruta(origen, destino)
        if not ruta:
            return math.inf
        return sum(
            distancia(a, b) for a, b in zip([origen, *ruta], ruta)
        )
```

Los consumidores piden una ruta o una posición válida sin implementar la búsqueda de caminos. Aquí abstracción significa ocultar el detalle del algoritmo tras una operación útil; Mapa no es una clase abstracta ABC. Nivel, personajes y ruido consultan la misma instancia para usar paredes coherentes.

Fuente: src/mapa.py, clase desde línea 82.

Personaje

Reúne posición, tamaño y movimiento de los personajes.

UML: recorte de la caja original



Aplicación de POO

Abstracción y herencia: base de Prisionero y Guardia.

Encapsulamiento: _x, _y y _velocidad son internos.

Asociación: guarda el Nivel que recibe.

Código del proyecto (fragmento)

```
class Personaje(ABC):

    def __init__(self, nivel, x, y, velocidad):
        self.nivel = nivel
        self._x = float(x)
        self._y = float(y)

    def get_posicion(self):
        return self._x, self._y

    @abstractmethod
    def update(self):
        pass
```

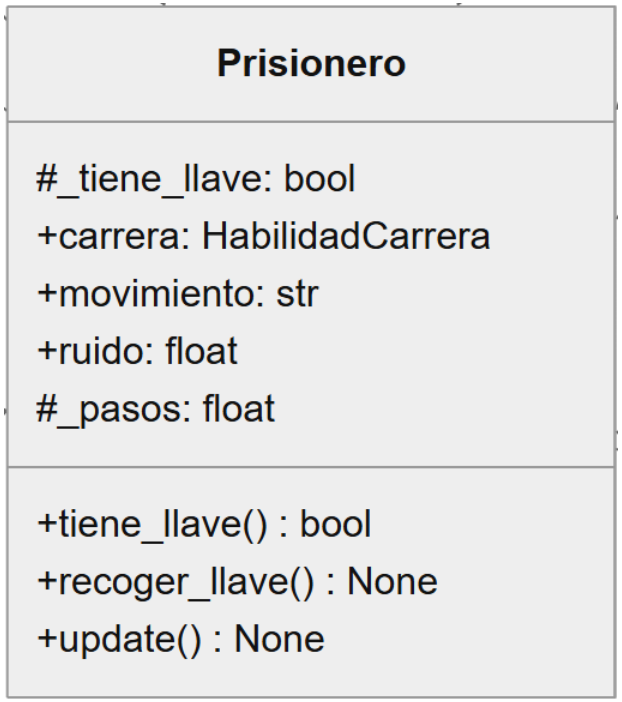
La clase abstracta reúne posición y movimiento común. get_posicion devuelve las coordenadas, mientras mover aplica las colisiones del mapa. update queda como contrato obligatorio para las clases hijas. El guion bajo señala uso interno por convención, no una prohibición de acceso del lenguaje.

Fuente: src/personajes.py, clase desde línea 39.

Prisionero

Es el personaje controlado por el jugador y el origen del ruido de pasos.

UML: recorte de la caja original



Aplicación de POO

Herencia y polimorfismo: implementa update de Personaje.

Composición: crea su HabilidadCarrera.

Encapsulamiento: controla _tiene_llave con métodos.

Código del proyecto (fragmento)

```
class Prisionero(Personaje):

    def __init__(self, nivel, x, y):
        super().__init__(nivel, x, y, 145)
        self.carrera = HabilidadCarrera()

    def tiene_llave(self):
        return self._tiene_llave

    def recoger_llave(self):
        self._tiene_llave = True
```

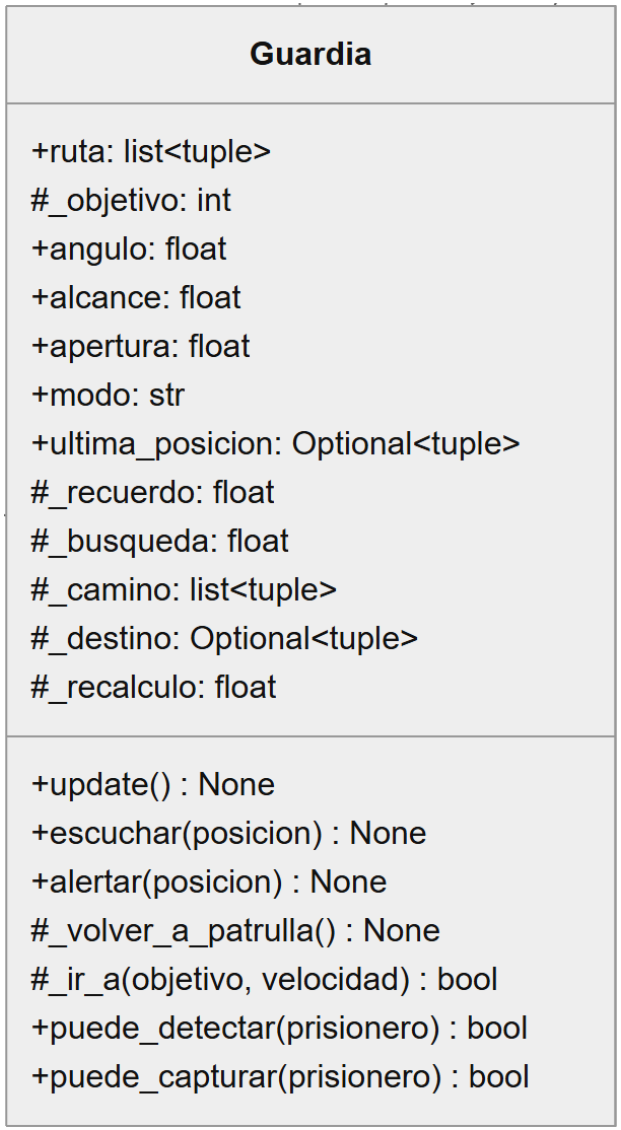
Reutiliza el movimiento de Personaje y define cómo responde al teclado. Su habilidad de carrera es una parte creada por él. Los métodos tiene_llave y recoger_llave concentran la consulta y el cambio del estado de la llave. Su update participa en la misma llamada polimórfica que el de Guardia.

Fuente: src/personajes.py, clase desde línea 58.

Guardia

Patrulla, escucha ruidos, persigue al jugador y lo captura por contacto.

UML: recorte de la caja original



Aplicación de POO

Herencia y polimorfismo: implementa update de Personaje.

Encapsulamiento: gestiona _objetivo y _camino.

Dependencia: consulta al Prisionero para detectarlo.

Código del proyecto (fragmento)

```
class Guardia(Personaje):

    def __init__(self, nivel, ruta):
        self._objetivo = 1
        self._camino = []

    def update(self):
        if self.puede_detectar(self.nivel.prisionero):
            self.alertar(self.nivel.prisionero.get_posicion())
        elif self.modo == "PERSIGUE" and self._recuerdo == 0:
            self.modo = "BUSCA"
```

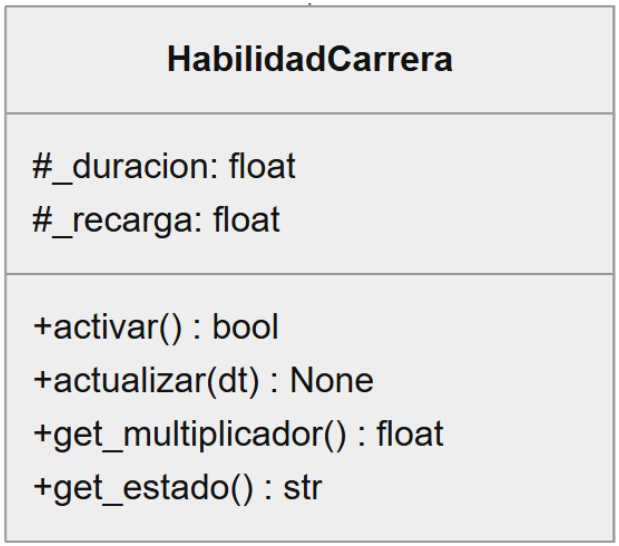
Comparte la base Personaje, pero decide una ruta y una conducta de vigilancia. Los atributos con guion bajo y métodos internos organizan su objetivo y camino. escuchar recibe las notificaciones del ruido. Sus modos son cadenas dentro de la clase; no forman la jerarquía de State usada por Juego.

Fuente: src/personajes.py, clase desde línea 97.

HabilidadCarrera

Controla el aumento temporal de velocidad del prisionero.

UML: recorte de la caja original



Aplicación de POO

Encapsulamiento: gestiona _duracion y _recarga.

Composición: es creada por cada Prisionero.

Código del proyecto (fragmento)

```
class HabilidadCarrera:

    def activar(self):
        if self._duracion > 0 or self._recarga > 0:
            return False
        self._duracion = 2.0
        return True

    def get_multiplicador(self):
        if self._duracion > 0:
            return 1.5
        return 1.0
```

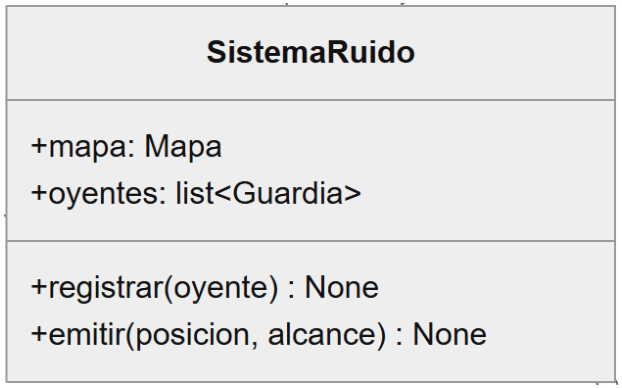
Concentra la duración, la recarga y las reglas de activación. Otros objetos piden activar o consultar el multiplicador sin repetir esas reglas. No hereda de Personaje: es una habilidad que el Prisionero contiene. Sus campos internos se identifican mediante el guion bajo.

Fuente: src/personajes.py, clase desde línea 6.

SistemaRuido

Emite los ruidos de pasos y avisa a los guardias que pueden escucharlos.

UML: recorte de la caja original



Aplicación de POO

Asociación: conserva el Mapa recibido.

Agregación: registra guardias creados por Nivel.

Notificación: avisa a los oyentes que pueden escuchar.

Código del proyecto (fragmento)

```
class SistemaRuido:

    def __init__(self, mapa):
        self.mapa = mapa
        self.oyentes = []

    def registrar(self, oyente):
        if oyente not in self.oyentes:
            self.oyentes.append(oyente)
```

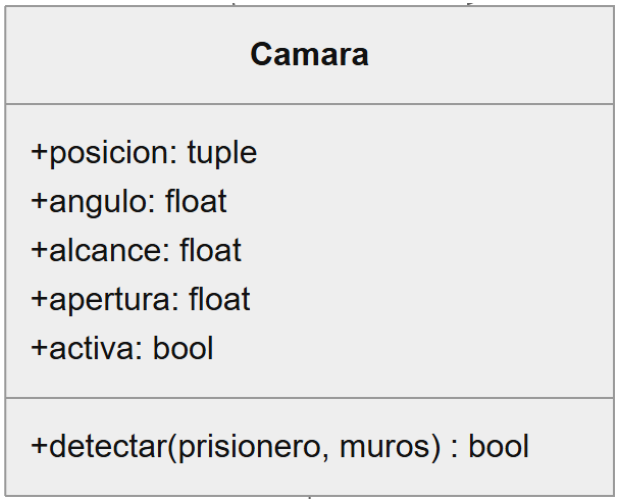
No construye el mapa ni los guardias: conserva referencias a objetos existentes. Usa la distancia de camino para decidir qué oyentes reciben un paso. Esta comunicación se parece a Observer, pero Guardia no hereda de ObservadorPuntaje. El ejemplo formal de ese patrón es Puntaje con PanelPuntaje.

Fuente: src/ruido.py, clase desde línea 4.

Camara

Comprueba si el prisionero está dentro de su campo de visión.

UML: recorte de la caja original



Aplicación de POO

Composición: Nivel construye sus cámaras.

Dependencia: detectar recibe Prisionero y muros.

Código del proyecto (fragmento)

```
class Camara:

    def detectar(self, prisionero, muros):
        return dentro_de_vision(
            self.posicion,
            prisionero.get_posicion(),
            self.angulo,
            self.alcance,
            self.apertura,
            muros,
            prisionero.radio,
        )
```

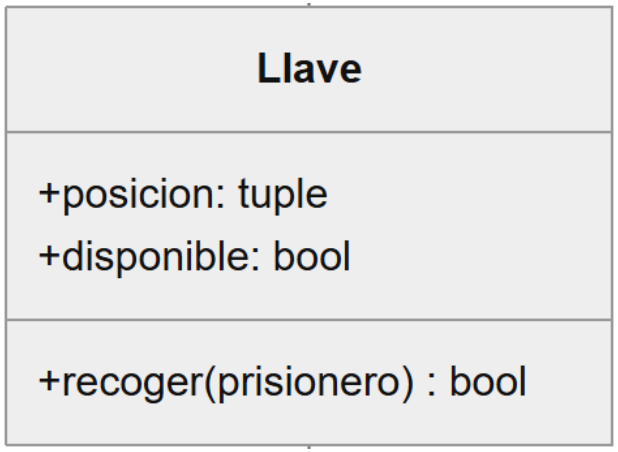
La cámara conserva su posición y campo de visión. detectar usa objetos recibidos para devolver si ve al jugador, sin guardarlo como atributo. Es dependencia de uso y no herencia de Personaje. Nivel interpreta el resultado y alerta a los guardias.

Fuente: src/nivel.py, clase desde línea 51.

Llave

Es el objeto que habilita la salida.

UML: recorte de la caja original



Aplicación de POO

Composición: cada Nivel crea su Llave.

Dependencia: recoger utiliza el Prisionero recibido.

Código del proyecto (fragmento)

```
class Llave:

    def recoger(self, prisionero):
        if (
            self.disponible
            and distancia(
                self.posicion, prisionero.get_posicion()
            )
            < 22
        ):
            self.disponible = False
            prisionero.recoger_llave()
            return True
        return False
```

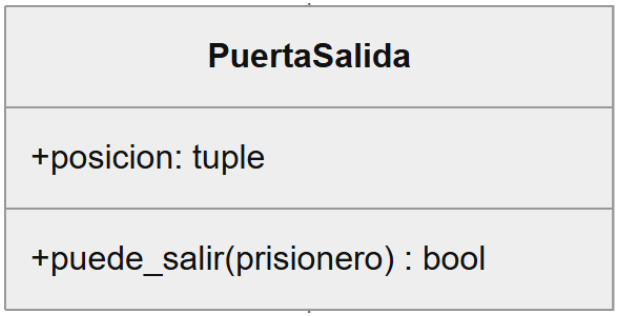
La llave reúne posición, disponibilidad y recogida. recoger consulta al prisionero y le comunica que obtuvo una llave, evitando concederla varias veces. El atributo disponible es público; el ejemplo de encapsulamiento controlado de la posesión está en Prisionero._tiene_llave.

Fuente: src/nivel.py, clase desde línea 64.

PuertaSalida

Representa el lugar por el que se termina un nivel.

UML: recorte de la caja original



Aplicación de POO

Composición: Nivel crea su PuertaSalida.

Dependencia: puede_salir consulta al Prisionero.

Código del proyecto (fragmento)

```
class PuertaSalida:

    def puede_salir(self, prisionero):
        return (
            prisionero.tiene_llave()
            and distancia(
                self.posicion, prisionero.get_posicion()
            )
            < 22
        )
```

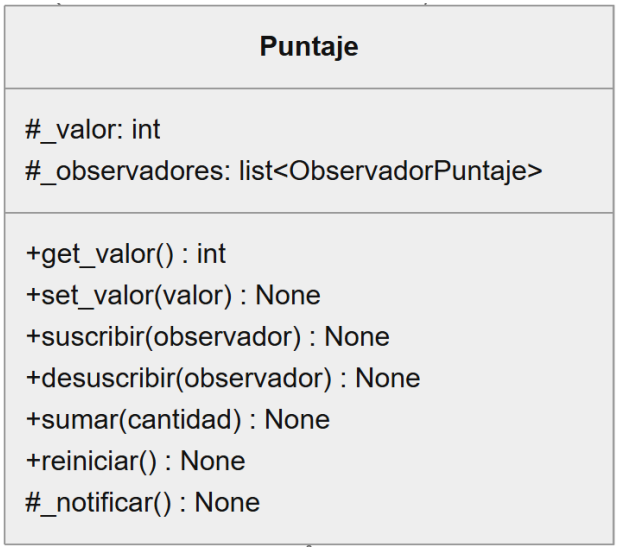
Agrupa la ubicación de la salida y la regla de acceso: estar cerca y tener la llave. El Prisionero se recibe como parámetro, no se crea ni se conserva en la puerta. Nivel decide después si el resultado completa una etapa o toda la campaña.

Fuente: src/nivel.py, clase desde línea 77.

Puntaje

Conserva los puntos y avisa cuando cambian.

UML: recorte de la caja original



Aplicación de POO

Encapsulamiento: valida los cambios de _valor.

Agregación: conserva observadores creados fuera.

Observer: notifica mediante actualizar_puntaje.

Código del proyecto (fragmento)

```
class Puntaje:

    def set_valor(self, valor):
        if type(valor) is not int or valor < 0:
            raise ValueError(
                "El puntaje debe ser un entero no negativo."
            )
        self._valor = valor
        self._notificar()

    def _notificar(self):
        for observador in tuple(self._observadores):
            observador.actualizar_puntaje(self._valor)
```

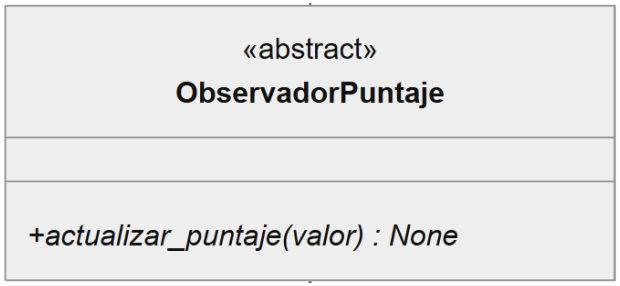
El getter permite consultar y el setter rechaza cualquier dato que no sea un entero no negativo. El cambio llama a _notificar, que usa la interfaz de los observadores. Puntaje no conoce cómo dibuja la interfaz. El mismo objeto también participa por agregación en Nivel.

Fuente: src/puntuacion.py, clase desde línea 18.

ObservadorPuntaje

Define cómo debe recibir los cambios un observador del puntaje.

UML: recorte de la caja original



Aplicación de POO

Abstracción: exige implementar actualizar_puntaje.

Herencia: es la base de PanelPuntaje.

Código del proyecto (fragmento)

```
class ObservadorPuntaje(ABC):

    @abstractmethod
    def actualizar_puntaje(self, valor):
        pass
```

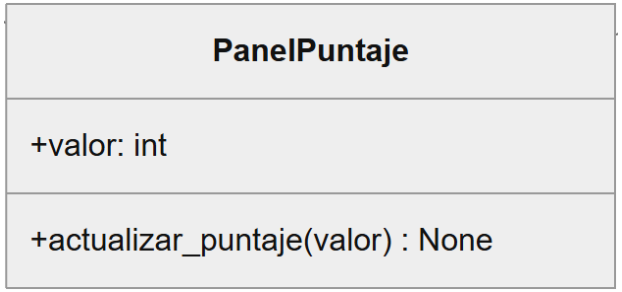
Define el contrato para recibir un valor de puntaje. abstractmethod obliga a las clases concretas a implementar esa operación. Separa lo que el emisor puede pedir de la implementación del receptor. Es el observador abstracto del patrón Observer.

Fuente: src/puntuacion.py, clase desde línea 4.

PanelPuntaje

Conserva el último puntaje notificado para mostrarlo en pantalla.

UML: recorte de la caja original



Aplicación de POO

Herencia: deriva de ObservadorPuntaje.

Polimorfismo: implementa actualizar_puntaje.

Observer: recibe el nuevo valor que luego dibuja Vista.

Código del proyecto (fragmento)

```
class PanelPuntaje(ObservadorPuntaje):

    def __init__(self):
        self.valor = 0

    def actualizar_puntaje(self, valor):
        self.valor = valor
```

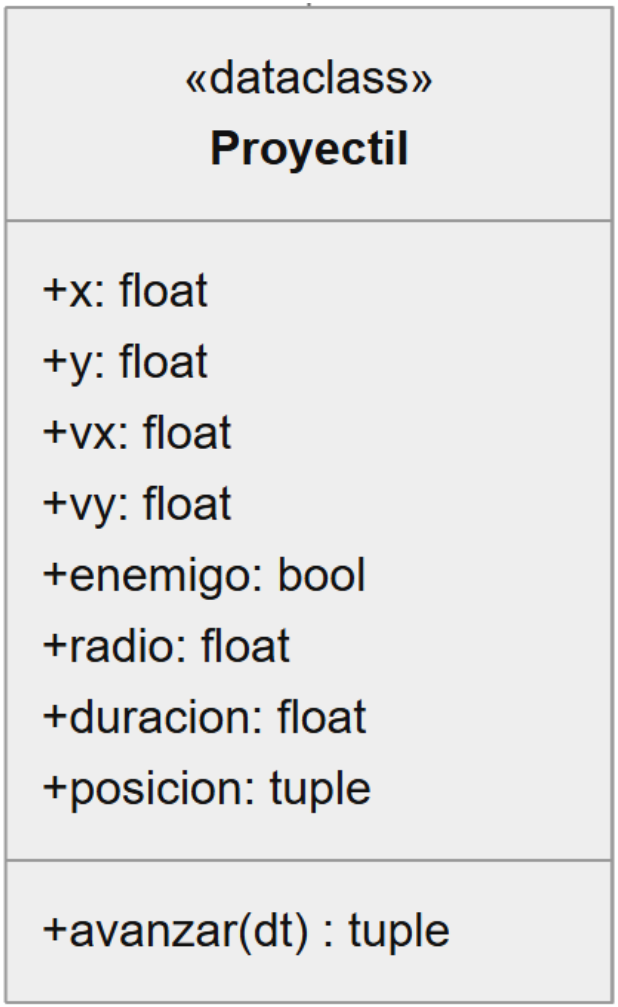
Puntaje puede notificarlo mediante la interfaz común de observador. La implementación concreta guarda el valor recibido. Su atributo valor es público: aquí el ejemplo principal es herencia y polimorfismo, no un campo protegido con setter. Juego lo registra mediante suscribir.

Fuente: src/puntuacion.py, clase desde línea 10.

Proyectil

Guarda la posición, velocidad, tamaño y duración de un disparo.

UML: recorte de la caja original



Aplicación de POO

Composición: Combate crea y conserva los disparos.

Dataclass: genera el constructor de sus datos.

Código del proyecto (fragmento)

```
@dataclass
class Proyectil:
    x: float
    y: float
    vx: float
    vy: float

    def avanzar(self, dt):
        inicio = self.posicion
        self.x += self.vx * dt
        self.y += self.vy * dt
        self.duracion -= dt
        return inicio, self.posicion
```

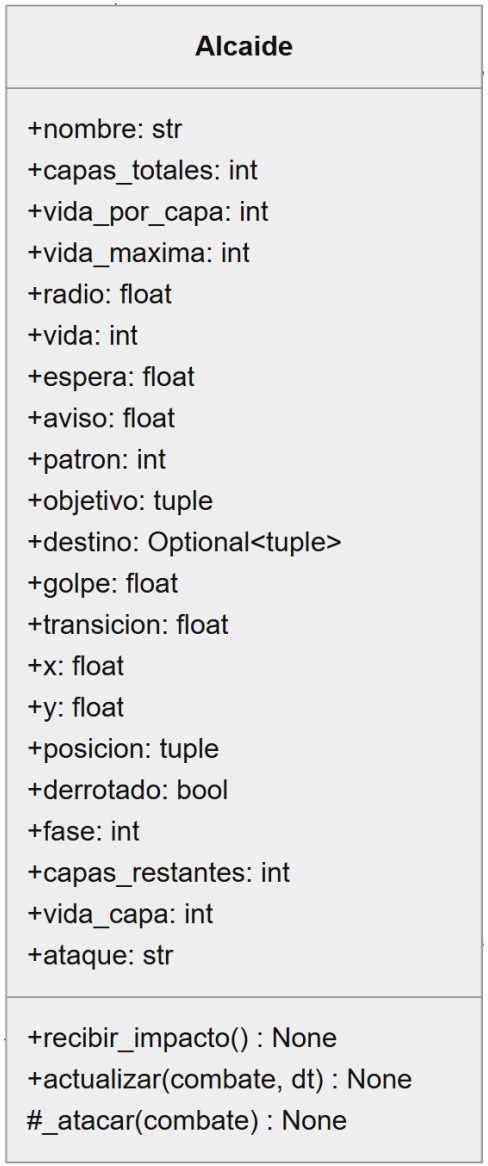
Agrupa posición, velocidad y duración con las operaciones del disparo. avanzar devuelve el segmento recorrido para comprobar impactos. Dataclass es una herramienta del lenguaje, no uno de los seis fundamentos de la rúbrica. Sus campos son públicos y no hereda de otra clase del juego.

Fuente: src/combate.py, clase desde línea 8.

Alcaide

Controla al jefe del último nivel.

UML: recorte de la caja original



Aplicación de POO

Composición: Combate crea su Alcaide.

Dependencia: actualizar recibe el Combate.

Propiedades: consultan fase, posición y derrota.

Código del proyecto (fragmento)

```
class Alcaide:

    @property
    def posicion(self):
        return self.x, self.y

    @property
    def derrotado(self):
        return self.vida <= 0
```

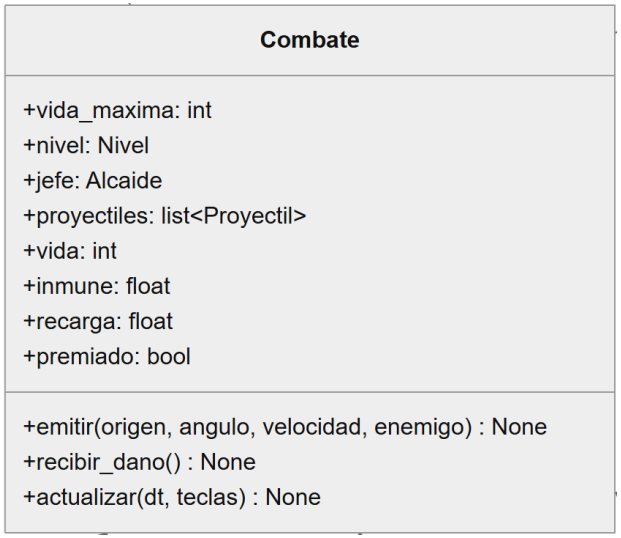
Concentra los ataques y las reglas del jefe. Se comunica con el Combate recibido para emitir disparos o consultar el mapa. Las propiedades calculan vistas del estado, por ejemplo si ya fue derrotado. Su vida es pública, por lo que no debe presentarse como un atributo privado protegido por un setter.

Fuente: src/combate.py, clase desde línea 47.

Combate

Resuelve los ataques, los proyectiles y la vida en la arena.

UML: recorte de la caja original



Aplicación de POO

Composición: crea Alcaide y Proyectil.

Asociación: conserva el Nivel recibido.

Código del proyecto (fragmento)

```
class Combate:
    def __init__(self, nivel):
        self.nivel = nivel
        self.jefe = Alcaide()
        self.proyectiles = []

    def emitir(self, origen, angulo, velocidad, enemigo=True):
        self.proyectiles.append(Proyectil(
            *origen, math.cos(angulo) * velocidad,
            math.sin(angulo) * velocidad, enemigo,
            6 if enemigo else 5))
```

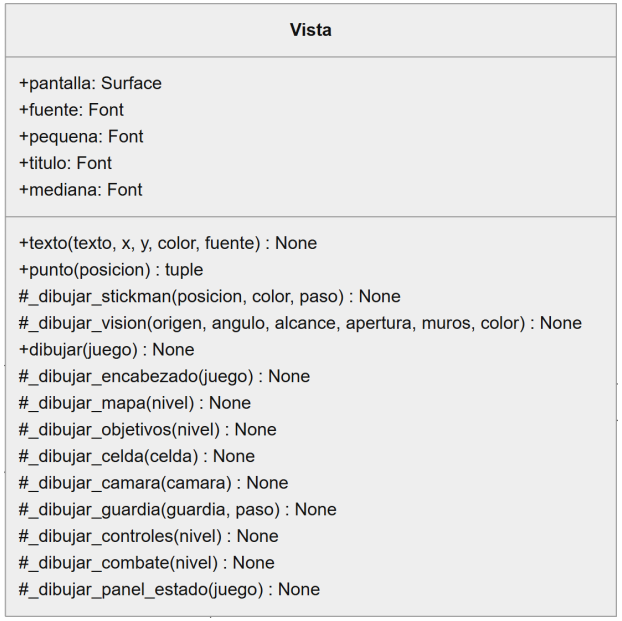
Es una parte que Nivel crea solo para la arena. A su vez construye el jefe y mantiene los proyectiles. Guarda una referencia a Nivel para acceder al jugador y al mapa compartido. Separa las reglas del combate del dibujo que realiza Vista.

Fuente: src/combate.py, clase desde línea 162.

Vista

Dibuja la interfaz, los mapas y los personajes stickman con Pygame.

UML: recorte de la caja original



Aplicación de POO

Asociación: conserva la pantalla que recibe.

Dependencia: dibujar consulta Juego y Nivel.

Código del proyecto (fragmento)

```
class Vista:

    def dibujar(self, juego):
        nivel = juego.nivel
        self._dibujar_encabezado(juego)
        self._dibujar_mapa(nivel)
        self._dibujar_objetivos(nivel)
```

Recibe Juego para leer sus datos y dibujarlos con Pygame. No decide las colisiones, la captura ni el puntaje. Separar estas responsabilidades permite probar las reglas sin abrir una ventana. Su constructor recibe la pantalla; no hereda de Personaje ni de una clase propia de sprites.

Fuente: src/vista.py, clase desde línea 12.